

M06 / PROTECTED ANSWERS

M06 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
YOUR STANDALONE REVIEW BOOK	3
Open only after a saved attempt.	3
Contents	3
Review DE06-01 Verify WSL2 and choose a safe Linux workspace	3
Review DE06-02 Linux navigation and safe file operations	4
Review DE06-03 Pipes, redirection, command history and inspection	4
Review DE06-04 Permissions, packages, sudo and command safety	4
Review DE06-05 Git state: init, status, add, commit, diff and log	4
Review DE06-06 Branches, merges and real conflict resolution	5
Review DE06-07 GitHub remotes: push, fetch, pull and tracking	5
Review DE06-08 Pull requests and review workflow	5
Review DE06-09 Python virtual environments inside a repository	5
Review DE06-10 .gitignore, secrets and repository hygiene	5
Review DE06-11 restore, revert, reflog and safe history repair	6
Review DE06-12 Shell scripts, exit codes and environment-driven commands	6
Review DE06-13 Repository handoff and clean-clone reproducibility	6

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

YOUR STANDALONE REVIEW BOOK

Open only after a saved attempt.

Use this book to diagnose a saved first attempt. Compare reasoning, observed state and controls rather than wording. Record the weakness, close this review, then solve the changed retry independently. Work done while review is open is repair evidence, not mastery evidence.

Contents

Review DE06-01 Verify WSL2 and choose a safe Linux workspace	3
Review DE06-02 Linux navigation and safe file operations	4
Review DE06-03 Pipes, redirection, command history and inspection	4
Review DE06-04 Permissions, packages, sudo and command safety	5
Review DE06-05 Git state: init, status, add, commit, diff and log	5
Review DE06-06 Branches, merges and real conflict resolution	6
Review DE06-07 GitHub remotes: push, fetch, pull and tracking	6
Review DE06-08 Pull requests and review workflow	7
Review DE06-09 Python virtual environments inside a repository	7
Review DE06-10 .gitignore, secrets and repository hygiene	8
Review DE06-11 restore, revert, reflog and safe history repair	8
Review DE06-12 Shell scripts, exit codes and environment-driven commands	9
Review DE06-13 Repository handoff and clean-clone reproducibility	9

OPEN ONLY AFTER A GENUINE ATTEMPT. Compare reasoning and observable state, not word-for-word prose. After repair, close this review and solve the changed retry.

Review DE06-01 | Verify WSL2 and choose a safe Linux workspace

A correct attempt names the actual environment rather than saying only "terminal." WSL2 is the subsystem architecture; Ubuntu is a distro; Bash is a shell. If your repo lives under /mnt/c, that is not automatically wrong, but you must know you chose the mounted Windows filesystem. For this course, prefer your Linux home for Linux-heavy repos unless you have a reason otherwise.

Changed retry: Fresh shell + new workspace path; translate a Windows path to /mnt/c form and explain the boundary.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-02 | Linux navigation and safe file operations

The most important repair is location awareness. If you cannot predict the target of `cp/mv/rm`, stop and use `pwd/ls` first. `rm` is not "move to Recycle Bin." Quoting matters when a path contains spaces. A safe learner proves the target before deleting it.

Changed retry: New source/ready tree; use one absolute and one relative path; prove copy vs move.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-03 | Pipes, redirection, command history and inspection

| transports stdout between processes. `>` replaces the output file; `>>` appends. `2>` targets stderr. If your daily output doubled because you used `append` on a regeneration job, the terminal syntax became a data-quality defect.

Changed retry: New text input; filter/count to stdout file and redirect a separate failure to stderr file.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-04 | Permissions, packages, sudo and command safety

Permission denied is evidence, not a command suggestion. Inspect owner/mode/path first. `chmod u+x script.sh` is a narrow change. `chmod 777` gives write/execute broadly and hides the real reasoning. `sudo` should be justified by system-level authority, not used to overpower project mistakes.

Changed retry: Second script: remove/restore only owner execute and prove direct-execution behavior.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-05 | Git state: init, status, add, commit, diff and log

The key model is working tree -> index/staging -> commit. `git add` selects the current version for the next commit; later edits are not magically restaged. `git diff` and `git diff --staged` answer different questions. GitHub has not appeared yet.

Changed retry: Stage a file, edit it again, commit, and prove the later unstaged edit remains.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-06 | Branches, merges and real conflict resolution

A branch is a movable pointer, not a second full project folder. A conflict means Git needs a human content decision. Resolving by deleting markers without explaining the intended final behavior is not mastery.

Changed retry: Different README run-command conflict; resolve according to actual project interface.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-07 | GitHub remotes: push, fetch, pull and tracking

origin is just a remote name. Fetch updates your knowledge/remote-tracking refs but does not merge into the current branch. Pull combines fetch with integration. If you used only a local bare remote, that proves Git mechanics but is not evidence that GitHub authentication/push worked.

Changed retry: Two clones + local bare remote: fetch in clone B and prove worktree does not change until integration.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-08 | Pull requests and review workflow

A PR is a hosted review proposal. The strongest evidence is a focused diff, rationale, validation and response to feedback. A solo self-review can be honest portfolio evidence, but it is not peer review. Offline diff practice must not be labelled a GitHub PR.

Changed retry: Changed PR/review scenario containing accidental .env; block merge and prove the proposed diff is repaired.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-09 | Python virtual environments inside a repository

.venv is generated/disposable project environment state. It should be easy to recreate and should not be tracked. If the project only runs because of global packages, the environment contract is incomplete.

Changed retry: Second repo: recreate .venv from scratch and run project without copying environment directory.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-10 | .gitignore, secrets and repository hygiene

Ignoring affects untracked matching paths; it does not purge tracked history. If a real secret was exposed, rotate/revoke it. Removing the latest file alone does not restore confidentiality. Do not use real credentials in this exercise.

Changed retry: Track fake demo-secret once; ignore is insufficient; stop current tracking and prove history still contains it.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-11 | restore, revert, reflog and safe history repair

Choose repair based on state. Restore can discard uncommitted file changes. Revert records a new inverse commit and is friendly to shared history. Reflog is local evidence of recent reference/HEAD movement that can help locate commits. Avoid destructive history rewriting as a reflex.

Changed retry: New bad shared-style commit: revert it; then create a recovery branch from a reflog-discovered commit.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-12 | Shell scripts, exit codes and environment-driven commands

The caller sees an exit code, not your intentions. Print an error to stderr and return non-zero for failure. Quote path variables. Environment variables are configuration boundaries, and secret values should not be echoed into logs.

Changed retry: New shipment wrapper: positional input + OUTPUT_DIR; test path with spaces and distinct exit codes.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.

Review DE06-13 | Repository handoff and clean-clone reproducibility

The clean clone is the truth test. If README-only setup cannot reproduce the result, the handoff is incomplete. Hidden data, global packages, undocumented env vars and absolute personal paths are exactly what this test is designed to expose.

Changed retry: Fresh shipment repository with changed branch/conflict/revert/wrapper rules and a new clean-clone proof.

Move-on rule: the changed task works with this review closed and you can explain one realistic failure + one independent validation.